

Evolution of Computers

1 Introduction

I used to wonder how 0s and 1s of computer become a picture, a movie, a book or text and how the idea of computer came to human mind. What discoveries, advancements, and inventions led to the birth of computers, computing machines, or a machine to do calculation?

As the definition of a computer is to compute or do calculation, the earliest idea of computers emerged from the need for machines that could help in calculations. So it was a calculator earlier and now made for doing other jobs like document preparation, streaming videos, watching movies, and data and record keeping. Internet usage bloomed and enhanced the usage and capabilities of computers. The journey from a bulky computer of room size capable of only doing certain calculations to making it programmable and small enough to fit in your palm has come a long way.

So the story starts from a machine to calculate, i.e. a calculator. They evolved slowly from manual tools for counting, to mechanical devices, to electronic machines, and finally to the intelligent digital systems we use today.

2 Human Computers

Earlier, humans were called computers, usually ladies who were deployed to do the calculation work. The word *computer* may be inspired from them. The logic behind working of computers is also highly inspired by these human computers, as they were given a set of instructions and data. They would look at the instructions to understand what operations had to be performed on the data. That is what our computers do.

3 Early Calculating Devices

In ancient times, Chinese used the abacus machine. There was also an instrument called Napier's bones for doing multiplications and divisions. Then came the mechanical era, where gears and levers were used for calculating.

4 Mechanical Era

The mechanical era started with “Pascaline” that could perform addition and subtraction. Gottfried Leibniz made “Reckoner,” another mechanical calculator to improve the Pascaline. This calculator could perform add, multiply, subtract, and divide. The basic underlying mechanism for both was similar.

4.1 Blaise Pascal’s Pascaline (1642)

French mathematician Blaise Pascal invented the Pascaline, one of the first mechanical calculators. It used a series of gears and wheels to perform addition and subtraction. He made the calculator to help his father, a tax collector, perform long addition and subtraction accurately and quickly. The machine was too bulky and was not a big success economically. Pascaline was built majorly for addition only; subtraction happened as 9’s complement.

4.2 Gottfried Leibniz’s Step Reckoner (1694)

German philosopher and mathematician Gottfried Wilhelm Leibniz improved upon Pascal’s design by creating the Step Reckoner, also called the Stepped Drum. This machine could perform all four arithmetic operations—addition, subtraction, multiplication, and division—using a special cylinder with teeth of increasing length. It could multiply as a series of additions and divide as a series of subtractions.

The above two machines are great at doing arithmetic calculations automatically. But they can’t yet be called computers.

They could calculate but not compute, and they were not programmable. In the 1700s and 1800s, the industrial age brought highly produced machines that were used to perform repetitive work. This led to the idea that if we can use machines to do logical operations as well, and if they can follow instructions, then they can repeat and automate tasks.

Two inventions highly influenced this idea: Jacquard’s loom and Charles Babbage’s Difference and Analytical Engines.

4.3 Jacquard’s Loom (1804)

Jacquard’s loom, an invention from the textile industry, paved the way for punched cards. It was a loom that wove patterns in cloth automatically, controlled by punched cards. A sequence of cards was used to weave or print a certain design or pattern on the cloth. Each card represented a pattern of holes, like 0s and 1s. If there was a hole, a hook could pass through it and lift a thread; if there was no hole, the hook stayed down. Several cards were chained together so that the loom could repeat and change patterns automatically.

Babbage took this idea of punch cards to store and control information and input.

Note

Jacquard's loom proved that instructions could be stored and reused using a punched card system. Charles Babbage adopted this concept for his Analytical Engine (1830s), the first design for a programmable computer. Later, Herman Hollerith used punch cards for data tabulation in the 1890 U.S. Census, leading to IBM's founding. Thus, the concept of programmability and data storage originated from a textile machine and became the backbone of early computing. Before usage of punch cards, calculations were based on the decimal system and punched cards paved way for binary representation, where a hole in a card can be 1 and no hole can be 0.

4.4 Charles Babbage's Difference Engine (1822)

British mathematician Charles Babbage is often called the "father of the computer." He designed the Difference Engine to automatically compute polynomial functions and print mathematical tables. He observed that mathematics tables used for navigation and astronomy were computed by hand and were heavily inconsistent because of human errors. He wanted to remove the human errors so that the calculations could be accurately done. He decided to build a machine to automate the calculation and printing of these tables.

The machine was designed to automatically calculate and print polynomial functions, used in logarithmic or trigonometric tables, by the method of finite differences. This method allows polynomial functions to be computed using only addition and subtraction, without multiplication or division. It was supposed to print mathematical tables automatically, without human typesetting. This was automation of calculation and printing, revolutionary for the 1820s.

He originally thought of making the machine powered by steam; however, it was changed to hand crank. The machine was never built because of funding issues, Babbage kept changing plans, and the machinery and technology were not advanced enough in the early 1800s.

4.5 Analytical Engine (1837)

By the 1830s, Babbage realized the Difference Engine was too limited because it could only do one specific kind of calculation, i.e. polynomials. So he worked on improving his Difference Engine and made it into a revolutionary design that can be considered analogous to modern day computers. This machine was called the Analytical Engine.

He envisioned a machine that could compute any algorithm given the right instructions. This led to the concept of programmability.

Ada Lovelace (1843) read Charles Babbage's work and recognized that machines could go beyond just pure calculations and could manipulate symbols according to rules, describing the concept of programming. She wrote a program for computing Bernoulli numbers using loops and conditions—100 years before the first actual computers. That is why she is often called the first programmer.

Although the Analytical Engine was never built because of technical limitations, the machine anticipated every major element of modern computers:

- Memory (store) for holding numbers and data
- Processor (mill) for performing calculations
- Input/output (punch cards)
- Program control (punch cards, loops, conditionals)

Essentially, it was the first blueprint of a modern computer, in the 1830s.

The mechanical era demonstrated that calculation could be automated and that machines could follow predetermined instructions. Babbage's genius was far ahead of its time. His idea was too complex for Victorian engineers to design with extreme precision. So his designs were largely forgotten for decades, but the concepts — memory, control, arithmetic, input/output — remained important in intellectual circles.

5 Electromechanical Era

Then electrical engineering began advancing in the late 1800s, and people started thinking: what if we use electricity to control machines instead of pure mechanics? That is where the electromechanical age begins.

5.1 Herman Hollerith's Tabulating Machine (1890)

American inventor Herman Hollerith created a punched card system to process data for the 1890 U.S. Census. His machine could read information encoded on cards and tabulate results much faster than manual methods. This innovation reduced census processing time from years to months and led to the founding of a company that would eventually become IBM.

How it worked: data such as age, gender, and job was punched into cards using a special machine. These cards were fed into Hollerith's Tabulating Machine. The machine used electrical connections to read the holes and count or sort the data automatically.

Hollerith's system had three main parts:

- a) Card Puncher — a typewriter-like machine used by operators to punch holes in blank cards.
- b) Tabulating Machine — the heart of the system, an electro-mechanical computer of the time. It had a metal plate with electrical pins that could drop into the holes. Wires connecting those pins led to counters or registers.
- c) Sorter Machine — after counting, another machine sorted cards automatically into boxes.

A punched card was placed on the machine's bed. The operator closed the top, pressing small metal pins downward. Wherever there was a hole, the pin went through and touched a pool of mercury or a metal contact underneath. This completed an electric circuit, sending a small current through the wire. That current triggered a counter to increase by one. When done, the machine ejected the card, and the next one was inserted.

5.2 Differential Analyser (1930)

Differential Analyser was developed by engineer Vannevar Bush. It was an electromechanical analog computing device used for solving differential equations.

5.3 Harvard Mark I (1944)

Harvard Mark I was another electromechanical, relay based computer. It was a collaboration between Harvard University and IBM. It worked on the decimal system. It was a general purpose programmable computer and later inspired electronic computers like ENIAC and EDVAC.

5.4 Zuse Z1 (1936)

Zuse Z1 was the first mechanical binary programmable computer. It had components like mechanical memory, arithmetic unit, and punch cards. The machine could store 64 words, 22 bits each, using levers or rods in two positions (0 or 1). It could perform addition, subtraction, multiplication, and division. Numbers were entered via punch cards or keyboard and converted to binary.

5.5 Zuse Z3 (1941)

Zuse Z3 was an electromechanical computer built using relays. A relay is an electrically controlled mechanical switch. When a small current flows through a coil, it creates a magnetic field, pulls a metal arm, and connects or disconnects another circuit. So it can represent binary states (on/off), just like a transistor or a bit of data. That made relays perfect for logic and computation before semiconductors existed. Z3 was the first working programmable computer which performed general purpose arithmetic, stored data in memory, and executed instructions automatically.

The electromechanical era of computing used mechanical parts controlled by motors and relays. It had many limitations: speed was slow, wear and tear was an issue, it produced a lot of noise, and it was bulky. Because of these limitations, engineers searched for a purely electronic switch, with no moving parts. By the mid-1940s, wartime code breaking and ballistics computation demanded faster and more reliable machines. Scientists looked toward radio and amplifier technology, which already used vacuum tubes for rapid signal control. That is where the electronic era began.

6 Electronic Era

The electromechanical era proved that a machine could act as a general-purpose processor, but it hit a physical wall: friction and inertia. A mechanical relay could only switch a few times per second before its metal arms wore out, sparked, or fell out of sync. To solve the complex mathematics of wartime ballistics and code breaking in the 1940s, humanity had to eliminate moving parts entirely. They turned to the vacuum tube.

6.1 Vacuum Tubes

A vacuum tube, or thermionic valve, was essentially a modified light bulb that acted as a high-speed electronic switch. By heating a filament inside a vacuum, it emitted electrons toward a metal plate. By introducing a small control grid between them, engineers could apply a tiny voltage to completely block or allow the flow of electricity.

Because it controlled a stream of pure electrons rather than moving a physical piece of metal, a vacuum tube could switch thousands of times faster than a relay. It had no moving parts, made no noise, and could represent binary states instantaneously.

6.2 Colossus (1943)

Built in Britain by engineer Tommy Flowers, Colossus was the world's first electronic digital programmable device. It used roughly 1,500 vacuum tubes to perform boolean logical operations to crack the complex Lorenz ciphers used by the German high command. It proved that vacuum tubes could handle massive logical data streams reliably under pressure.

6.3 ENIAC (1945)

The Electronic Numerical Integrator and Computer, ENIAC, built at the University of Pennsylvania, was the first general-purpose, fully electronic digital computer. It was a room-sized titan containing nearly 18,000 vacuum tubes, weighing 30 tons, and consuming enough electricity to power a small town.

The limitation was that ENIAC was incredibly fast, performing 5,000 additions per second, but it lacked an internal memory for programs. To program ENIAC, operators had to physically pull, reroute, and plug in thousands of patch cables and set mechanical switches manually. Changing a program took days.

6.4 The Stored-Program Concept

Mathematician John von Neumann saw ENIAC's reprogramming nightmare and proposed a revolutionary conceptual leap: the stored-program concept. He argued that a computer should not separate instructions from data. Instead, both data and the instructions that manipulate that data should be stored together in the same electronic memory space.

This blueprint, known as the Von Neumann Architecture, divided the computer into distinct functional units:

- Control Unit
- Arithmetic Logic Unit (ALU)
- Memory Unit

This architecture birthed EDVAC (1949) and UNIVAC I (1951), the first commercial computer. Computers were finally truly programmable; to change a task, you simply loaded a different set of 0s and 1s into memory.

6.5 The Vacuum Tube Dilemma

While revolutionary, vacuum tubes created a massive scalability wall. They generated immense heat, consumed massive amounts of power, and burned out constantly. On a bad day, ENIAC experienced a tube blowout every few hours, requiring engineers to hunt through a maze of hot glass to find the broken switch. Computers could not get smaller or more complex because adding more tubes meant the machine would cook itself.

7 Transistor Era

In December 1947, John Bardeen, Walter Brattain, and William Shockley at Bell Labs invented the point-contact transistor, changing human history forever.

Instead of boiling electrons across a vacuum gap, the transistor utilized the quantum properties of semiconductors, primarily Germanium at first and then Silicon. By sandwiching a tiny piece of specially treated silicon between two metal contacts and applying a microscopic electrical charge to a third “gate” terminal, they could alter the semiconductor’s ability to conduct electricity.

The transistor was a solid-state switch. It had no filament, generated almost no heat, consumed a fraction of the power, was shockproof, and most importantly, it was microscopic. It could switch millions of times per second.

By the late 1950s and early 1960s, second-generation computers like the IBM 7090 replaced thousands of fragile vacuum tubes with sleek, reliable transistors. Computers shrunk from the size of entire houses to the size of a few filing cabinets, while their reliability shot up from hours to weeks.

8 Integrated Circuits and Microprocessors

While transistors solved the reliability crisis, they ran into a new manufacturing bottleneck in the late 1950s: the tyranny of numbers. Engineers needed hundreds of thousands of individual transistors, resistors, and capacitors. Every component had to be manufactured

separately, placed on a circuit board, and soldered together by hand. A single loose wire or bad solder joint could ruin the entire machine.

8.1 Integrated Circuit

The solution came independently from Jack Kilby at Texas Instruments and Robert Noyce at Fairchild Semiconductor. They asked: if transistors, resistors, and wires are all made of silicon and metal, why not manufacture them all on the same piece of silicon?

This was the integrated circuit, or microchip. Circuits were now printed using photolithography, shining ultraviolet light through architectural stencils onto a chemically treated silicon wafer to etch thousands of transistors and their connecting pathways at once.

8.2 Moore's Law

In 1965, Gordon Moore noticed that the number of transistors engineers could cram onto a single square inch of silicon was doubling roughly every two years. This was not a law of physics, but it became an industry guide that drove computing down a path of exponential cost reduction and power scaling.

8.3 Microprocessor (1971)

Even with integrated circuits, chips were still custom built for specific tasks. In 1969, a Japanese calculator company named Busicom asked Intel to design 12 distinct custom chips for a new printing calculator line. Ted Hoff realized that designing 12 separate chips was wasteful. Instead, he proposed a single, generalized chip that could do any logical operation, governed by software instructions stored in a separate memory chip.

In 1971, Intel released the Intel 4004, the world's first commercial microprocessor. It packed an entire central processing unit onto a single silicon chip smaller than a thumbnail. It had 2,300 transistors and matched the computing power of the 30-ton ENIAC.

9 Modern Architectural Advancements

As transistors continued to shrink throughout the 1980s and 1990s, computer architects focused on making the flow of instructions highly efficient. They hit two major architectural barriers: the frequency wall and the memory wall.

9.1 Pipelining

In early computers, the CPU fetched an instruction, waited for it to decode, waited for it to execute, and only then fetched the next one. Architects introduced pipelining — treating instruction execution like a factory assembly line. While Instruction 1 is executing

in the ALU, Instruction 2 is being decoded, and Instruction 3 is already being fetched from memory.

9.2 Out-of-Order Execution

To prevent the assembly line from stalling due to data delays, architects developed control logic that could execute independent instructions out of order. CPUs like the Intel Pentium Pro (1995) began analyzing incoming code streams dynamically. If Instruction 2 was waiting for a slow data delivery from memory, the CPU would look ahead, find Instruction 3, and execute it out of order if it was independent, then put the results back in a neat sequence later.

9.3 Branch Prediction

When encountering an if-else condition in code, the CPU would guess which path the program would take and start executing it ahead of time. If it guessed correctly, the computer felt incredibly fast; if it guessed wrong, it threw away the work and restarted. That penalty was small compared to the performance gains.

9.4 Caching and the Memory Wall

By the 1990s, CPU clock speeds were accelerating exponentially, but the speed of main memory was lagging far behind. The CPU was running at lightning speed but spent much of its time waiting for data from slow DRAM. This became known as the memory wall.

Architects solved this by embedding a small layer of incredibly fast, expensive memory called SRAM cache directly onto the CPU die. The cache predicted what data the CPU would need next based on proximity and historical usage, acting as a buffer so the CPU rarely had to travel all the way out to main memory.

10 GPU and AI Era

By the mid-2000s, the computing world hit the ultimate physical boundary: the power wall. For decades, shrinking a transistor made it naturally faster and less power hungry. But at very small sizes, transistors began leaking current. Raw clock-speed acceleration was dead.

10.1 Multicore Shift

If you cannot make one brain tick faster, you must put more brains on the same chip. The industry pivoted to multicore processors.

10.2 GPU

A CPU core is highly optimized and sequential, while a GPU is built for massive data parallelism. For rendering, video, and matrix-heavy workloads, the GPU completely outmatched the CPU.

The programmable GPU era, along with CUDA, enabled thousands of tiny, simple ALU cores working in parallel.

10.3 Deep Learning and Specialization

The breakthrough in deep learning came when researchers realized that the same matrix operations used in graphics were also needed for neural networks. The modern AI revolution was powered not by faster CPUs, but by highly parallel graphics hardware.

As we enter the 2020s, computers increasingly use heterogeneous computing—a collection of highly specialized chips on the same system or even the same piece of silicon.

Compute Type	Flexibility	Efficiency	Ideal Use Case
CPU	Highest	Lowest	Operating systems, web browsing, general logic
GPU	Medium	Medium-High	3D rendering, video encoding, AI training
FPGA	High	High	Prototyping, high-frequency trading, signal routing
ASIC	Zero	Highest	TPUs, neural engines, Bitcoin mining

10.4 FPGAs

An FPGA is a chip that contains millions of open look-up tables and programmable connections. Instead of writing software that runs on a fixed CPU, an engineer writes code that physically alters the internal electrical pathways of the FPGA. If the algorithm changes, you do not throw away the chip; you flash a new bitstream to rewire it.

10.5 ASICs and NPUs

If an algorithm is perfectly optimized and stable, companies design ASICs, chips custom-designed for one specific task. The prime example is Google's TPU or Apple's neural hardware. These chips use architectures like systolic arrays, where data flows through the silicon like a continuous wave, calculating matrix algebra directly without fetching instructions or hitting memory caches.

11 Conclusion

Think back to where this journey began.

It started with human computers reading text instructions to manipulate numerical records on sheets of paper. We mechanized it into gears, automated it with textile punch-card patterns, translated it into pure streams of electrons with vacuum tubes, and printed it globally onto microchips.

Today, billions of silicon switches sit inside devices that fit in our pockets and power our digital world. The 0s and 1s that once barely calculated tax records now shift across specialized CPU, GPU, and NPU architectures, manipulating data at mind-boggling scale and parallel speed. The calculator did not just automate data; it became the canvas upon which modern civilization is built.